

Data In, Answers Out

CSV → Kafka → Neo4j → Chatbot Pipeline

A complete, working data pipeline that takes an arbitrary CSV file uploaded through a web UI, streams every row through Apache Kafka, loads it into a Neo4j graph database, and answers natural-language questions about that data through a **deterministic, rule-based chatbot** - with no LLM required at runtime. Every answer is either backed by a real Cypher query against Neo4j, or is an honest refusal - the system never invents data that isn't in the uploaded file.

Full Project Report

1. High-Level Architecture

Every uploaded CSV flows through five real services, wired together by Docker Compose: the UI, the FastAPI backend, Kafka (running in KRaft mode, no ZooKeeper), a standalone Loader consumer process, and Neo4j 5.x Community. Note how the API never writes row data to Neo4j directly - it only publishes to Kafka; the Loader is the only component permitted to write CSV row content into the graph.

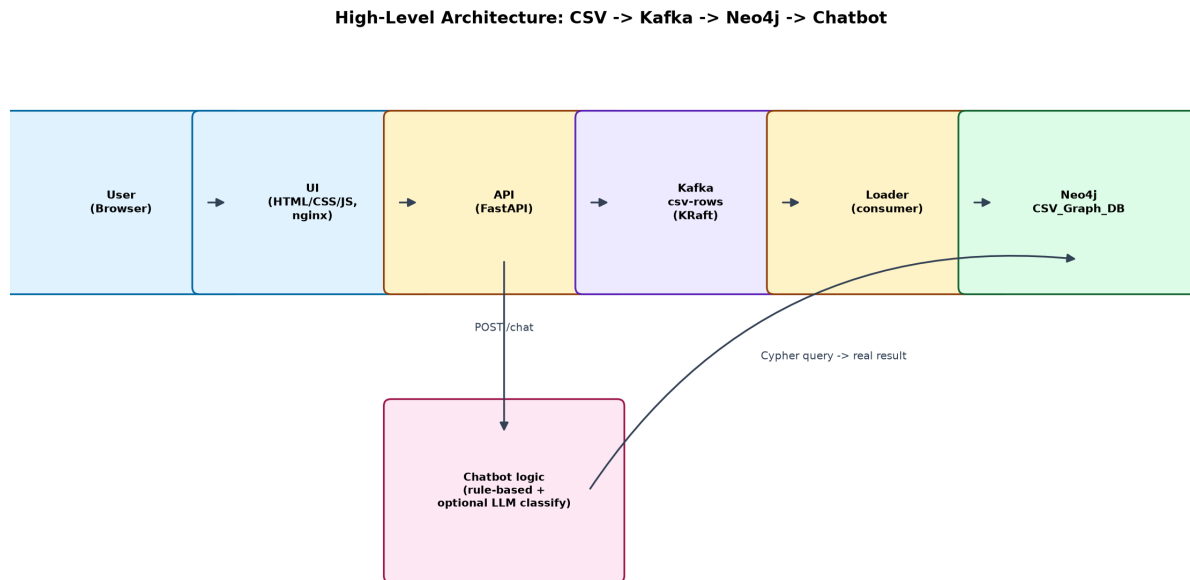


Figure 1 - High-level service architecture and data flow.

Why Kafka sits in the middle

The API publishes each row as a Kafka message and returns immediately; the Loader consumes those messages independently. A slow or temporarily unavailable Neo4j can never block a CSV upload, and the ingestion pipeline can be restarted or scaled without touching the API.

2. Detailed Ingestion Sequence

The sequence below traces one CSV upload from the browser all the way to a live, continuously-updating progress readout - every step here is a real network call, not a simulated one.

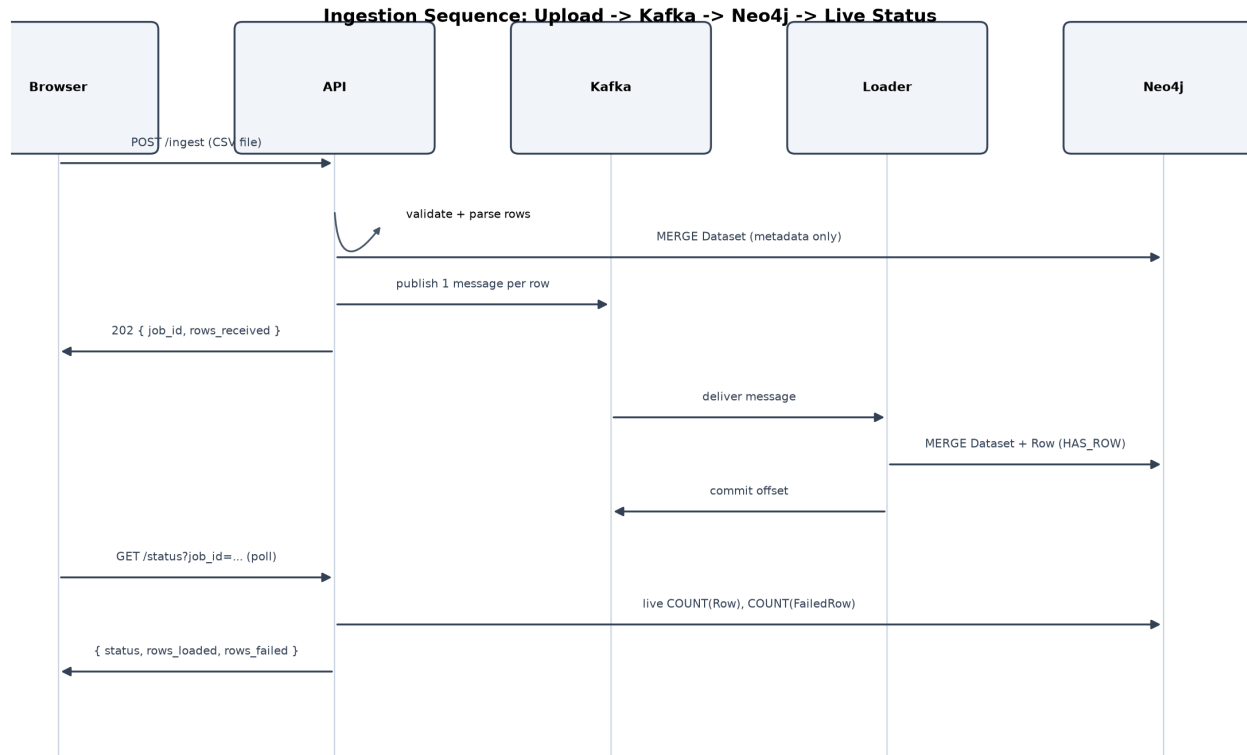


Figure 2 - Full request/response sequence for a CSV upload.

Key design decisions in this flow

Decision	Why
dataset_id = sha256(filename + file bytes)	Re-uploading the exact same file always maps to the same dataset_id, so the Loader's MERGE is a true no-op the second time - this is what makes duplicate uploads idempotent.
Row key = dataset_id + '_' + row_index, MERGE never CREATE	Guarantees no duplicate nodes even if Kafka redelivers a message (crash/restart) or the same file is uploaded twice.
/status counts are live Cypher queries, never an incrementing counter	A counter could double-count on message replay; a live COUNT of real graph nodes cannot lie, and self-heals after any restart.
Failed rows written as (:FailedRow) nodes	rows_failed in /status is a real, queryable number, not a guess.
API never writes row/column data to Neo4j, only the Loader does	Satisfies the requirement that uploaded CSV data reaches Neo4j only through Kafka.

3. The Neo4j Graph Model

The model is intentionally generic (Dataset → Row) rather than a fixed schema, because the CSV structure is unknown ahead of time - any CSV with any columns can be uploaded, and every column becomes a property on the Row node.

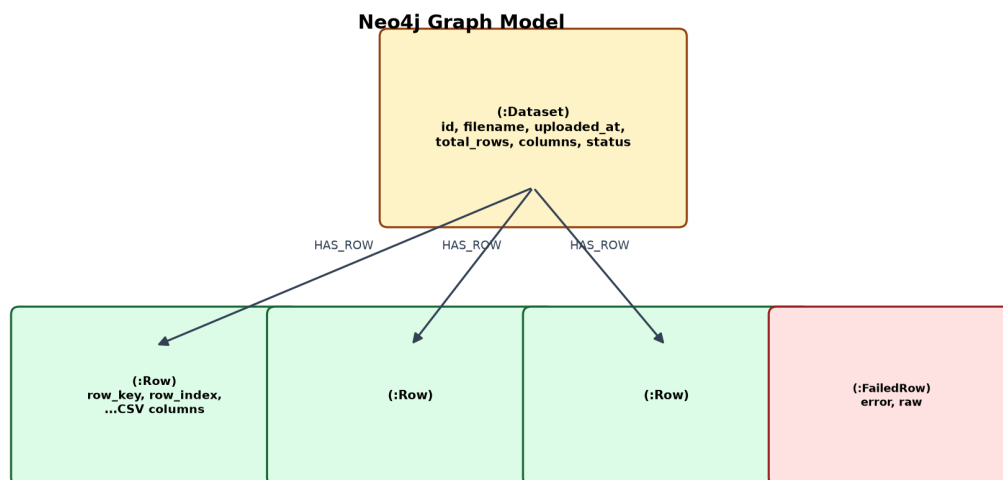


Figure 3 - Neo4j node/relationship model.

4. The Chatbot: Question to Cypher to Answer

The critical guarantee of this system: the optional LLM fallback (used only when the rule-based matcher fails, and only when an API key is configured) is allowed to pick *which* fixed Cypher template to run and *which* real column/value to plug into it - and every choice it makes is re-validated against the dataset's actual schema before use. It never writes free-form Cypher and never generates the answer text; the answer is always built in code from the literal Neo4j query result.

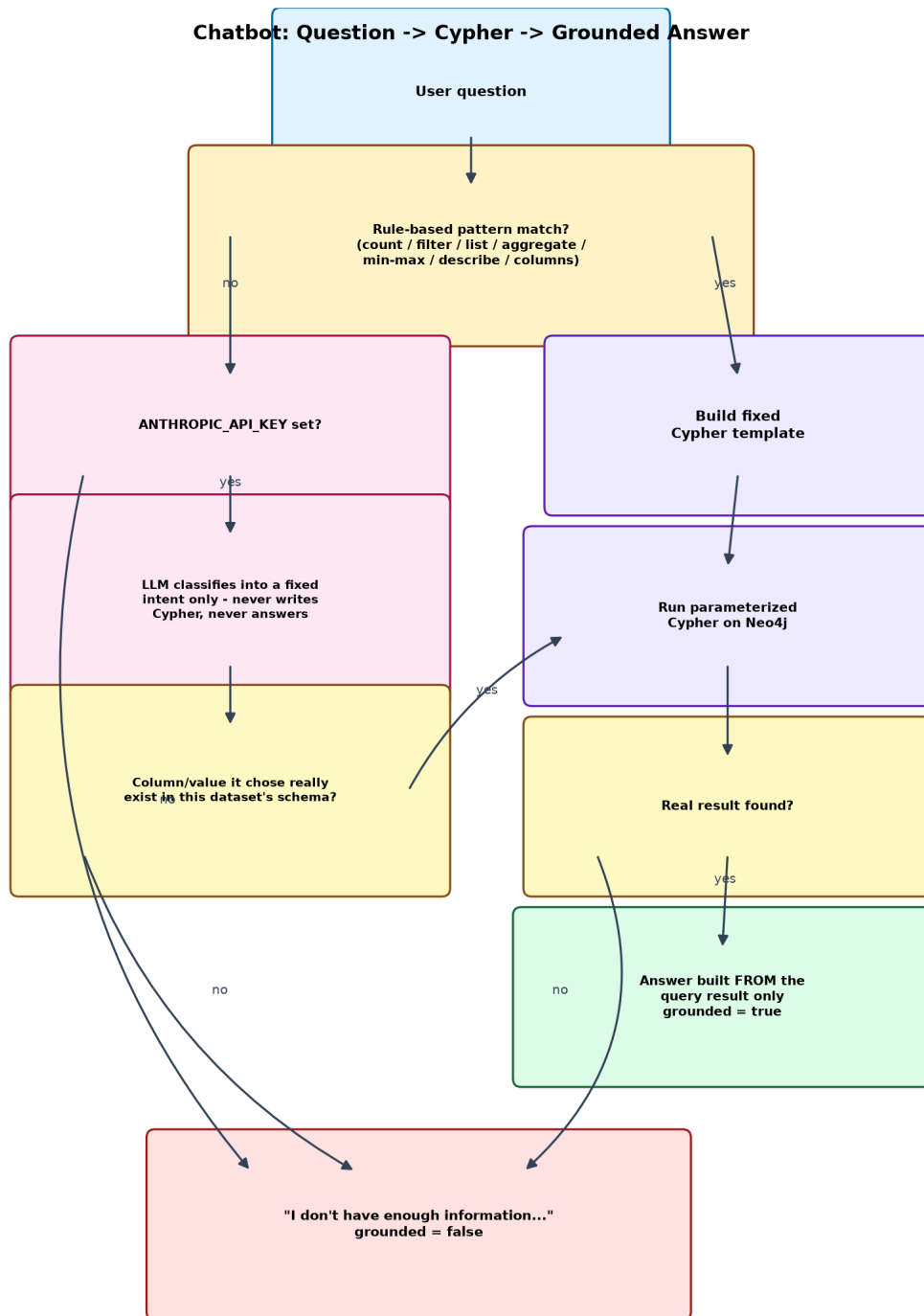


Figure 4 - Chatbot question-answering decision flow.

Supported question types (rule-based, no LLM needed)

Intent	Example question	Cypher shape
Count all rows	"How many rows are there?"	MATCH (r:Row) RETURN count(r)
Count filtered	"How many rows are in Billing?"	MATCH (r:Row {group:'Billing'}) RETURN count(r)
Show matching rows	"Show Billing records"	... WHERE r.group='Billing' RETURN r LIMIT 10
List unique values	"What groups are available?"	RETURN DISTINCT r.group
Aggregation	"What is the average age?"	RETURN avg(toFloat(r.age))
Min / max	"What is the maximum age?"	RETURN max(toFloat(r.age))
Schema	"How many columns are there?"	reads Dataset.columns
Describe	"Describe the dataset"	row count + column list

5. Component Breakdown

ui/ - vanilla HTML/CSS/JS, served by nginx

- **index.html** - dashboard, upload, ingestion status, graph explorer, chatbot, activity log, system info
- **app.js** - every API call is real; there is no hardcoded demo data anywhere in the UI

api/ - FastAPI

- **routes/ingest.py** - validate + parse CSV, publish to Kafka
- **routes/status.py** - live progress computed from Neo4j
- **routes/health.py** - real Kafka + Neo4j connectivity checks
- **routes/chat.py** - question in, grounded answer out
- **routes/insights.py** - Data Health Score, Suggested Questions, Graph sample
- **services/chatbot.py** - rule-based question-to-Cypher matching
- **services/llm_intent.py** - optional hybrid LLM classifier (off by default)
- **services/kafka_producer.py, neo4j_service.py, csv_utils.py** - infrastructure clients and validation

loader/ - Kafka consumer

- **consumer.py** - polls Kafka, commits offset per message
- **neo4j_loader.py** - MERGE Dataset/Row, records FailedRow on error

Infrastructure (docker-compose.yml)

- Kafka 3.7.0 in KRaft mode, single broker
- Neo4j 5.24 Community Edition

6. WOW Features (Beyond the Core Requirements)

- **Data Health Score** (GET /insights) - a 0-100 score computed from real column completeness and load completeness. Never a fabricated number.
- **Smart Suggested Questions** - generated dynamically from the dataset's actual columns, so suggestions always work against whatever was uploaded.
- **Graph Explorer** - a dependency-free, pure-SVG radial layout of the real Dataset and sampled Row nodes, with click-to-inspect real node properties. No third-party graph-visualization library was added, so it carries zero risk to the core pipeline's build.

7. Reliability & Error Handling

- **Startup readiness:** the API and Loader both retry connecting to Kafka and Neo4j on startup; docker-compose.yml uses real health checks (service_healthy).
- **Idempotent loading:** every write uses MERGE on a stable key - uploading the same file twice never creates duplicate nodes.
- **Input validation:** empty files, header-only files, and non-.csv files all return clean 400 responses with a specific message - never a raw 500.
- **Global exception safety net:** any unanticipated server error still returns clean JSON instead of a bare crash page (hardened after a real CSV with a stray \r character inside a text field triggered an uncaught csv.Error).
- **Large files:** nginx is configured for large request bodies and long timeouts - a 60,000-row / 7.5 MB CSV was verified to upload and fully load with zero failures.
- **Non-root containers, pinned image versions:** apache/kafka:3.7.0, neo4j:5.24-community - no 'latest' tags anywhere; api, loader, and ui containers all run as a dedicated non-root user.

8. Technology Stack

Layer	Technology
Frontend	Vanilla HTML / CSS / JavaScript (Tailwind via CDN), served by nginx
Backend API	Python 3.11, FastAPI
Message broker	Apache Kafka 3.7.0, KRaft mode, single broker
Database	Neo4j 5.24 Community Edition, official Python driver
Chatbot	Rule-based Cypher templates; optional hybrid LLM (Anthropic API) for question classification only
Containerization	Docker + Docker Compose

9. What Was Actually Verified

- Clean docker compose down -v && docker compose up --build from a fully empty volume - all 5 containers reach healthy/running with zero manual steps.
- GET /health returns real kafka_connected / neo4j_connected booleans backed by an actual Kafka metadata fetch and a Neo4j verify_connectivity() call.
- A 15-row CSV traced end-to-end: /ingest → Kafka topic csv-rows → Loader logs → confirmed present via cypher-shell directly against csv-graph-db.
- The same 15-row CSV uploaded twice: Neo4j still contains exactly 15 Row nodes.
- A 60,000-row / 7.5 MB CSV uploaded through the real UI: all 60,000 rows loaded, 0 failures.
- Chat tested for count-all, count-filtered, average, max, distinct-values, schema, and describe intents, plus deliberately off-topic and nonsense questions - every response carries the correct grounded flag.
- Empty CSV, header-only CSV, and a non-csv file all return clean 400s, never a 500.
- Full UI click-path exercised in a real browser: upload → live progress → dashboard KPIs/preview table populate from Neo4j → suggested-question chip clicked → grounded chat answer with real Cypher/result shown → Graph Explorer node click shows real row properties.

10. Known Limitations

- The rule-based chatbot cannot understand unlimited natural-language phrasing by design; the optional LLM fallback widens coverage but is still restricted to the same fixed set of query templates and never fabricates an answer.
- `/chat` and `/status` operate on one dataset at a time (the most recently uploaded, or an explicit `job_id`) - there is no cross-dataset querying.
- A Loader restart can incur a short (up to ~30-45s) delay before Kafka reassigns its consumer group partitions; this affects latency only, never correctness, since MERGE makes any replay safe.

11. How to Run It

```
git clone <this-repo>
cd <repo-folder>
cp .env.example .env    # fill in NEO4J_PASSWORD at minimum
docker compose up --build
```

Then open the UI (default `http://localhost:3001`) and: (1) upload a `.csv` file, (2) watch the live ingestion progress, (3) ask a question about the data in the chat panel, (4) explore the graph in the Graph Explorer tab.

See **REPORT.md** for the original design-decision log and **README.md** for the full project specification this implementation follows.